

기말 프로젝트 보고서

Django 기반 별자리·띠별 운세 제공 웹 서비스

구성원: 20230778 이지연, 20230789 배희겸

1. 프로젝트 목표

본 프로젝트의 목적은 Python과 Django 프레임워크를 활용하여 시드 기반 로직 기반으로 한 맞춤형 운세 서비스의 전체 구조를 이해하고 구현하는 것이다. 단순한 정적 정보 제공을 넘어, 사용자의 고유값(생년월일)과 서버의 시간 데이터를 결합하여 실시간으로 개인화된 결과를 생성하고 관리하는 웹 애플리케이션의 동작 원리를 체득하는 것을 목표로 한다.

본 주제는 파이썬 웹 프로그래밍 강의에서 학습한 Form 처리, 서버 측 로직 구성, 데이터베이스 연동, 템플릿 렌더링 과정을 하나의 서비스 흐름으로 종합적으로 적용할 수 있다는 점에서 적합하다고 판단하였다. 특히 운세 서비스는 사용자의 입력값에 따라 결과가 달라지는 구조를 자연스럽게 설계할 수 있어, 단순 CRUD 예제를 넘어 동적 콘텐츠 생성과 데이터 누적 관리가 결합된 웹 애플리케이션 구현에 적합한 주제라고 판단하였다.

구체적으로는 다음과 같은 목표를 설정하였다.

1. Django의 MVT 패턴을 기반으로 데이터 저장(Model), 비즈니스 로직(View), 사용자 인터페이스(Template)의 유기적 연결을 구현한다.
2. 입력 값과 날짜 정보를 활용한 시드 기반 로직을 적용하여, 외부 데이터 수집이나 크롤링에 의존하지 않고도 동적 콘텐츠를 생성하는 방식을 실습한다.
3. 사용자의 생년월일을 기준으로 데이터베이스(DB)에 히스토리를 누적하고, 과거 기록을 최신순으로 조회하는 데이터 관리 기능을 실습한다.
4. CSS 애니메이션 및 동적 그래프를 활용하여 단순 텍스트 위주의 예제를 넘어선 수준 높은 사용자 경험(UX)을 제공한다.

이를 위해 본 프로젝트에서는 별자리 운세와 띠별 운세를 제공하는 웹 서비스를 주제로 선정하였다.

2. 기술적 요소

2.1 전체 아키텍처 개요

본 프로젝트는 Django의 표준 MVT 구조를 기반으로 하며, 운세 생성과 관련된 로직을 View와 분리하기 위해 Utility 모듈을 활용하는 구조로 설계되었다.

각 구성 요소의 역할은 다음과 같다.

- Model: 사용자 생년월일별 운세 결과 및 행운 점수를 저장하는 히스토리 관리.
- View: 요청 처리, 시드 기반 운세 생성 호출 및 DB 트랜잭션 관리.
- Template: CSS 변수 기반 파스텔 UI, 행운 지수 게이지 그래프 및 과거 기록 리스트 렌더링.
- Utility: 타로 점성술 및 명리학 데이터셋 기반의 텍스트 조합 알고리즘 구현.

이와 같은 구조를 통해 기능별 역할 분리를 명확히 하고, 코드의 가독성과 유지보수성을 고려한 설계를 지향하였다.

2.2 데이터베이스(DB) 설계

기본 데이터베이스인 SQLite를 사용하여 사용자의 이용 기록을 체계적으로 관리한다. 기존의 단순 정보 저장 방식을 넘어 통계 및 시각화 데이터(Score)를 포함하도록 설계하였다.

운세 히스토리 모델

사용자가 운세를 확인할 때마다 그 결과값과 수치화된 점수를 영구 저장한다.

- birth_date: 사용자의 생년월일 (과거 기록 조회 키값)
- fortune_date: 운세를 생성한 날짜 (자동 생성)
- star_name / star_text / star_score: 별자리 이름, 타로 기반 상세 리포트, 행운 점수 (0~100)
- zodiac_name / zodiac_text / zodiac_score: 띠 이름, 명리 기반 상세 리포트, 기운 점수(0~100)

이 구조를 통해 사용자는 본인의 생년월일 입력만으로 과거에 뽑았던 운세 지수의 변화량을 확인할 수 있다.

2.3 Utility 모듈 설계

크롤링의 불안정성을 제거하기 위해 '시드 기반 텍스트 생성 엔진'으로 설계하였다. 고유 시드(Seed) 생성: `date.today()`와 `birth_date`를 조합하여, 동일 날짜 내에서는 일관된 결과를 제공하도록 설계되었다.

동서양 융합 리포트:

별자리: 타로 카드의 메이저 아르카나 상징을 이용한 영성적 분석 제공.

띠별: 사주의 오행(五行)과 십성(十星) 용어를 활용한 정통 명리학적 조언 제공.

모듈형 문장 조합: 서론-본론-조언으로 이어지는 긴 본문을 랜덤 조합하여 수만 가지의 고유 리포트를 생성한다.

코드의 가독성과 유지보수성을 향상시키고자 하였다.

2.4 View 및 URL 설계

함수 기반 뷰(FBV)를 사용하여 데이터의 흐름을 제어한다.

각 View는 다음과 같은 흐름으로 동작한다.

- **GET 요청:** 사용자의 생년월일을 입력받는 메인 폼 페이지 렌더링
- **POST 요청:**
 - 1) 입력된 날짜를 바탕으로 Utility 모듈에서 별자리/띠 운세 및 점수 산출.
 - 2) 생성된 데이터를 FortuneHistory 모델을 통해 DB에 즉시 저장.
 - 3) 해당 생년월일 기준 최근 기록을 조회

최종적으로 현재 운세 결과와 과거 히스토리 목록을 Template으로 전달하여 사용자에게 제공한다.

2.5 Template 및 시각화 요소

Template에서는 Django Template Language를 활용하여 서버에서 전달된 운세 결과와 점수 정보를 화면에 출력하였다. 점수 정보는 CSS 스타일을 활용한 게이지 형태의 시각적 요소로 표현하여, 텍스트 정보 외에도 직관적인 이해가 가능하도록 구성하였다. 또한 긴 운세 문장의 가독성을 고려하여 줄바꿈 처리 및 레이아웃 구성을 조정함으로써 사용자 경험을 개선하였다.

- 행운 지수 그래프: CSS transition과 linear-gradient를 활용하여 점수에 따라 동적으로 차오르는 게이지 바(Gauge Bar) 구현.
- 텍스트 렌더링: Django의 linebreaksbr 필터와 CSS의 white-space: pre-wrap을 결합하여 긴 상세 본문의 줄바꿈을 완벽히 보존.
- UI/UX: 카툰 스타일의 안내 말풍선(Chat Bubble)과 별자리/띠별 고유 기호 배지를 통해 친근한 디자인 구성.

3. 스크립트 상세 설명

본 프로젝트의 주요 스크립트 구성과 각 파일의 역할은 다음과 같다.

- 유틸리티 로직 (fortunes/utls.py): 운세 생성의 핵심 로직을 담당하는 모듈로, 사용자의 생년월일과 날짜 정보를 기반으로 시드 값을 설정하고, 이를 통해 일관성 있는 운세 문구와 점수를 생성한다. 별자리와 띠별 운세를 각각 처리하도록 함수가 분리되어 있으며, View 로직과 독립적으로 관리됨으로써 코드의 재사용성과 유지보수성을 높였다.
- 모델 설정 (fortunes/models.py): 사용자가 조회한 운세 결과를 저장하기 위한 데이터베이스 구조를 정의한다. 생년월일, 운세 생성 날짜, 별자리/띠별 운세 텍스트 및 점수 정보를 포함하여, 과거 운세 기록을 히스토리 형태로 누적 관리할 수 있도록 설계되었다.
- 뷰 로직 (fortunes/views.py): 클라이언트로부터 전달된 요청을 처리하는 역할을 수행하며, 입력된 생년월일을 기반으로 Utility 모듈을 호출하여 운세를 생성하고, 그 결과를 데이터베이스에 저장한다. 또한 생성된 점수 정보를 화면에 전달하여 시각화 요소(그래프)로 표현할 수 있도록 데이터 흐름을 제어한다.
- 템플릿 (fortunes/templates/fortunes/index.html): 서버에서 전달된 운세 결과, 점수 정보, 과거 히스토리 데이터를 사용자 화면에 렌더링하는 역할을 한다. CSS 스타일과 시각적 요소를 활용하여 운세 점수를 직관적으로 확인할 수 있도록 구성하였으며, 입력 폼과 결과 출력이 하나의 화면에서 자연스럽게 이루어지도록 설계되었다.

4. 사용 예시

본 웹 서비스의 사용 흐름은 다음과 같다.

1. 사용자가 웹 페이지에 접속한다.
2. 날짜를 입력한다.
3. 선택한 항목에 대한 운세 조회 요청이 서버로 전달된다.
4. 서버는 데이터베이스에서 해당 날짜의 운세를 조회한다.
5. 조회된 운세 결과가 HTML 페이지로 렌더링되어 사용자에게 제공된다.
6. 데이터가 존재하지 않을 경우 Utility 모듈을 통해 기본 운세 문구를 생성하여 제공한다.

이러한 흐름을 통해 사용자는 별도의 로그인이나 복잡한 절차 없이 간단히 운세 정보를 확인할 수 있다.



5. 피드백 및 한계점

본 프로젝트는 강의 범위 내에서 구현되었기 때문에 다음과 같은 한계점을 가진다.

1. 로그인 및 사용자 맞춤 기능이 구현되지 않았다.
2. 모바일 환경을 고려한 반응형 UI 설계가 미흡하다.

그러나 이러한 한계점은 프로젝트의 확장 가능성을 보여주는 요소로 판단되며, 향후 다음과 같은 개선 방향을 고려할 수 있다.

- 사용자 계정 기반 맞춤 운세 제공
- 외부 API 연동을 통한 실시간 운세 데이터 수집
- 반응형 웹 디자인 적용

6. 레퍼런스

- Django 공식 문서 (<https://docs.djangoproject.com>)
- Python Web Programming 강의 자료
- 오하아사 웹 페이지(운세 콘텐츠 및 기능 레퍼런스).